

Scalable Multi-Tier Loan Application on AWS

Hands-on cloud deployment • Networking • Scaling • Monitoring

PROJECT FOCUS

Deploy • Scale • Monitor

EC2 | ALB | Auto Scaling | VPC | RDS | S3 | CloudWatch | SNS

COMPUTE

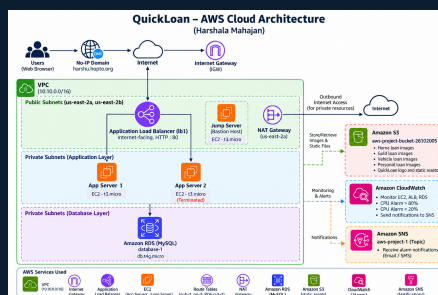
EC2 + Auto Scaling

NETWORK

VPC + ALB + NAT

OBSERVABILITY

CloudWatch + SNS



A Cloud Project by
Harshala Mahajan

Table of Contents

A concise map of the deployment evidence and project story.

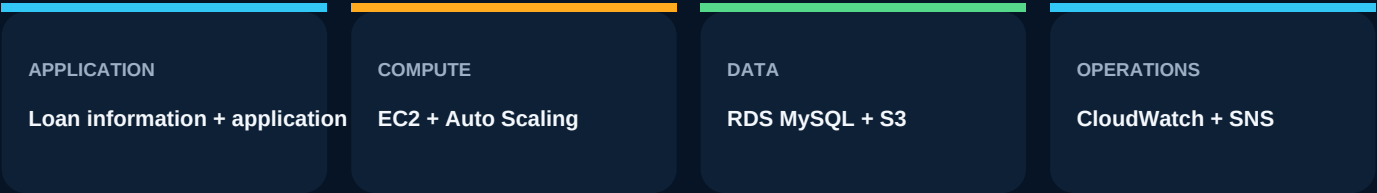
01	Project Overview Purpose, scope and architecture summary	03
02	Technology Stack Application, AWS services and deployment components	04
03	Architecture Diagram End-to-end QuickLoan AWS architecture	05
04	VPC & Networking VPC, subnets, route tables and NAT Gateway	06
05	Compute & Traffic EC2, AMI, Launch Template and ALB	07
06	Auto Scaling Auto Scaling Group and scaling configuration	08
07	Monitoring CloudWatch CPU alarms and operational signals	09
08	Data & Storage RDS MySQL and Amazon S3	10
09	Application QuickLoan application and user flow	11
10	DNS & Security No-IP routing and security-group model	12
11	Alerting Amazon SNS notification setup	13
12	AWS Service Map Services used across the solution	14
13	Testing & Validation Evidence-based checks and observations	15
14	Results & Takeaways What the project demonstrates	16
15	Future Scope Next improvements and production hardening	17

A working cloud-hosted loan application.

QuickLoan combines a user-facing website with AWS networking, compute, storage, scaling and monitoring.

QuickLoan is a web-based loan information and application system deployed on Amazon Web Services. The project demonstrates how a small web application can be hosted using a multi-service cloud environment rather than relying on a single standalone server.

The deployment uses a custom VPC, an Internet-facing Application Load Balancer, an Auto Scaling Group backed by a launch template, Amazon RDS/MySQL for the database tier, Amazon S3 for application assets, CloudWatch alarms for CPU monitoring and Amazon SNS for notifications.



PROJECT CONFIGURATION SNAPSHOT

Region	US East (Ohio) • us-east-2
VPC	vpc1 • 10.10.0.0/16
Load Balancer	lb1 • HTTP:80
Target Group	tg1 • Instance targets
Auto Scaling	asg-18sept • desired 1 • min 1 • max 5
Launch Template	lt1 • version 1
Database	database-1 • MySQL • db.t4g.micro
Storage	aws-project-bucket-26102005
DNS	harshu.hopto.org • CNAME to ALB

Application + AWS infrastructure.

The project stack is organized by responsibility so each service has a clear operational role.

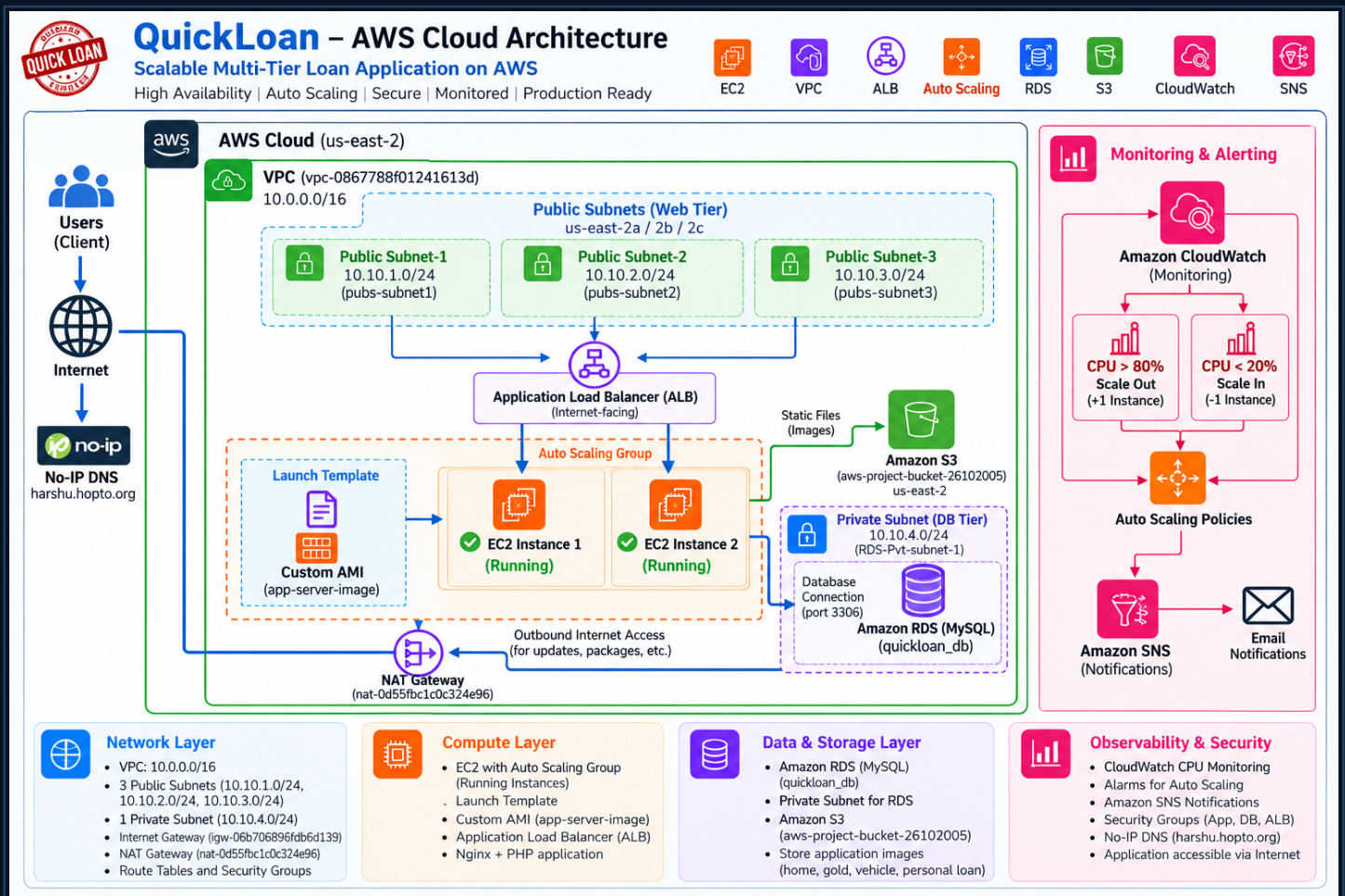
LAYER	TECHNOLOGY / SERVICE	PURPOSE
Frontend	HTML, CSS, JavaScript	User interface for the QuickLoan site
Application	QuickLoan web application	Loan information and application workflow
Compute	Amazon EC2	Runs the application workload
Load Balancing	Application Load Balancer	Internet-facing HTTP entry point
Scaling	Auto Scaling Group + Launch Template	Maintains configurable EC2 capacity
Networking	VPC, Subnets, Route Tables, IGW, NAT	Network segmentation and routing
Database	Amazon RDS – MySQL	Managed relational data tier
Object Storage	Amazon S3	Loan images, logo and static assets
Monitoring	Amazon CloudWatch	CPU metrics and alarm evaluation
Alerting	Amazon SNS	Notification topic/subscription
DNS	No-IP	harshu.hopto.org CNAME to ALB
Security	EC2 / DB Security Groups	Traffic control between layers

PROJECT STACK

AWS • Linux • Web Application • MySQL • CloudWatch • SNS • No-IP

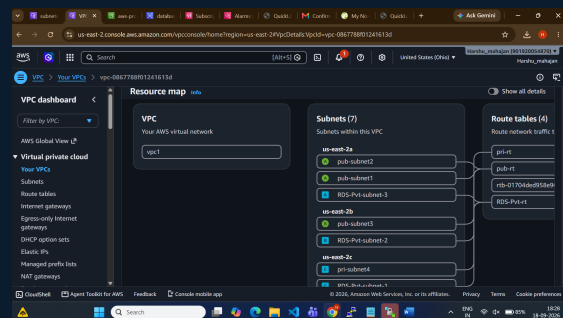
QuickLoan AWS Cloud Architecture

Updated after project completion: EC2 application instances are shown as RUNNING. No terminated server state is represented.

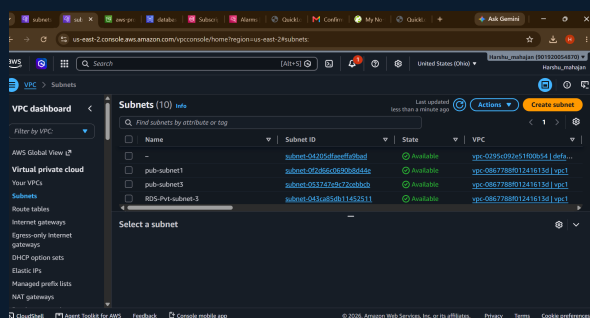


A structured VPC foundation.

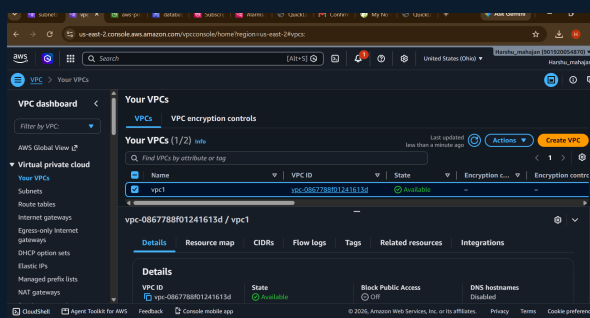
The supplied AWS evidence shows the VPC, subnet layout, route tables and NAT Gateway used by the deployment.



Custom VPC • vpc1 • 10.10.0.0/16



Subnet layout



Route tables

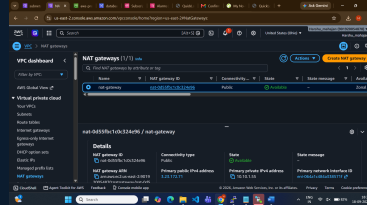
NETWORK PATH

Internet Gateway → public routing → load balancer; private routing supports internal resources.

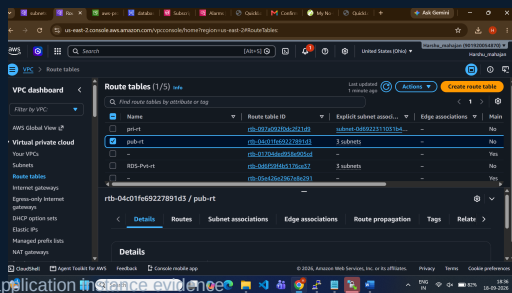
Scale the application layer.

EC2, the launch template, target group and Application Load Balancer form the application traffic path.

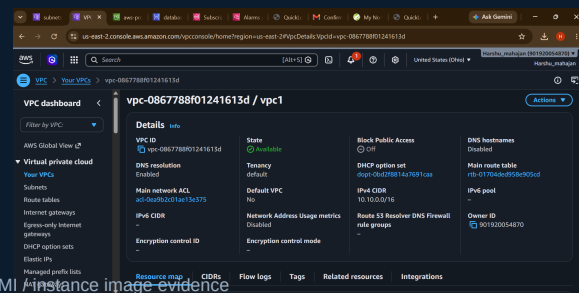
NAT Gateway evidence



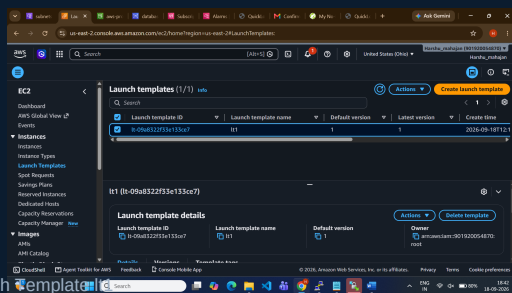
EC2 application instance evidence



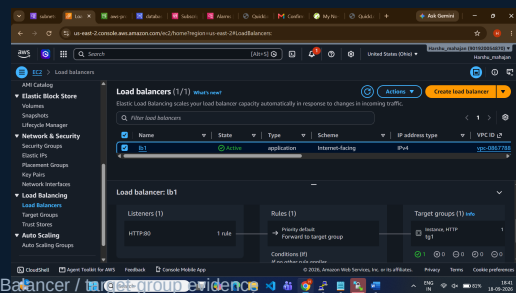
AMI / instance image evidence



Launch Template evidence

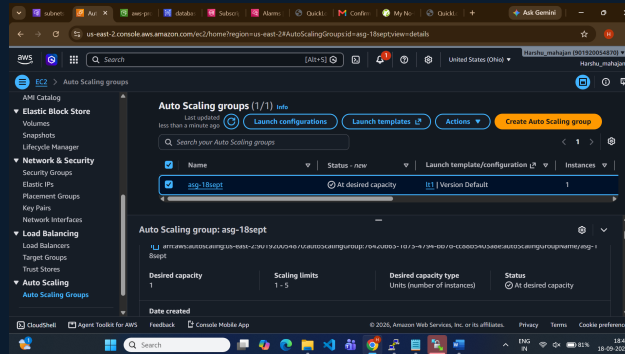


Load Balancer / target group evidence

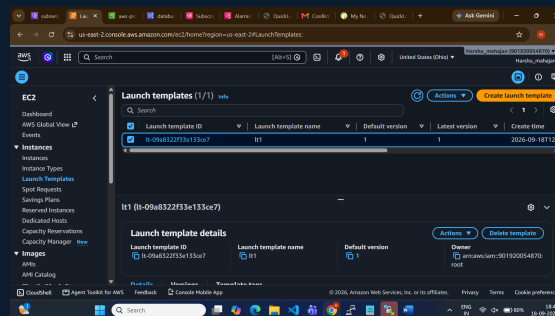


Capacity responds to workload signals.

The Auto Scaling configuration uses launch template lt1 and a capacity range of 1–5 instances.



Auto Scaling Group • asg-18sept



Launch Template • lt1 • version 1

CAPACITY

Desired: 1

Minimum: 1 • Maximum: 5

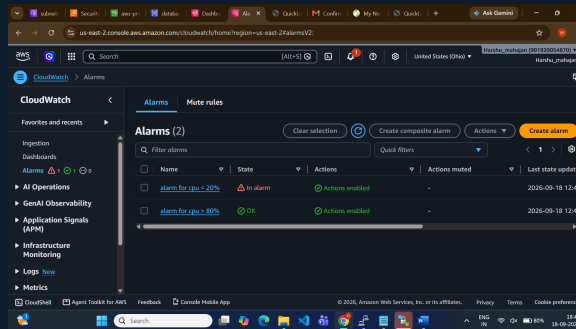
POLICY SIGNALS

CPU > 80% → scale-out signal

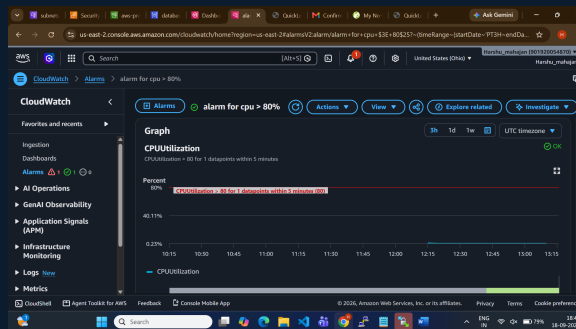
CPU < 20% → scale-in signal

CloudWatch signals the operations layer.

Two CPU threshold alarms are visible in the supplied evidence.



CloudWatch alarm list



CPU threshold alarm detail

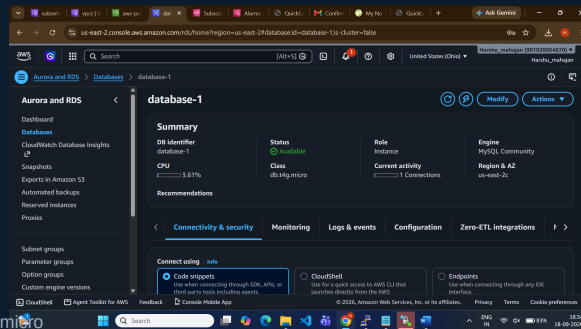
MONITORING LOGIC

EC2 CPU metric → CloudWatch alarm → operational response / SNS notification

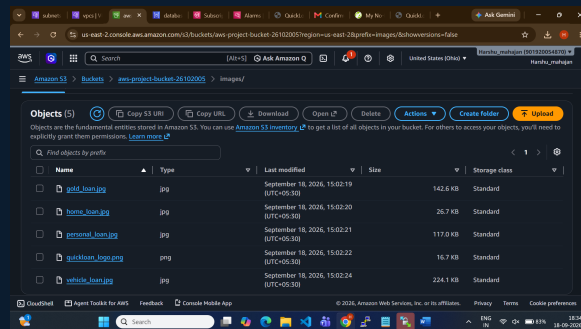
High-utilization threshold: >80% • Low-utilization threshold: <20%

Managed database. Object storage.

Application data and static assets are separated into dedicated AWS services.



Amazon RDS • database-1 • MySQL • db.t4g.micro



Amazon S3 • loan images / static assets

DATA FLOW

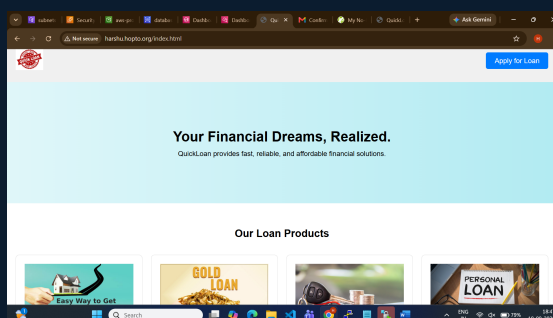
Application layer ↔ RDS/MySQL for relational data

Application assets → S3 bucket aws-project-bucket-26102005

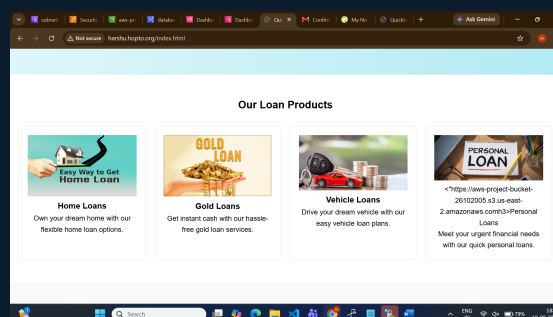
The cloud stack serves a working application.

The supplied screenshots show the public QuickLoan interface and the loan application workflow.

QuickLoan homepage



Loan product section



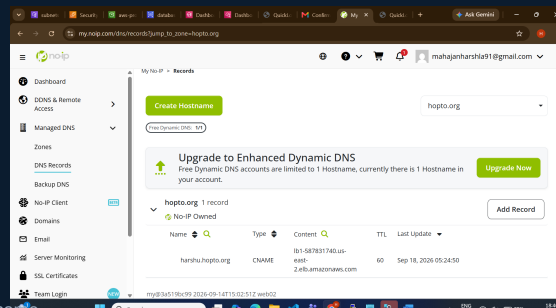
Apply for Loan form

VISIBLE USER FLOW

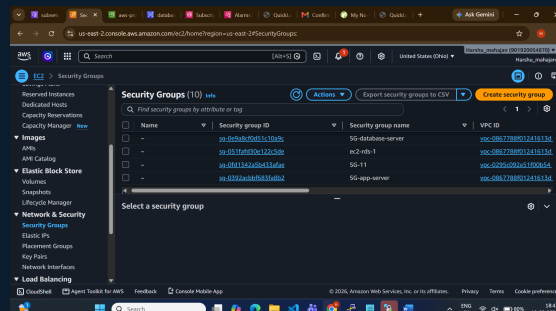
Homepage → Loan category → Apply for Loan → form submission controls

Public routing with layered access control.

No-IP provides the project hostname while security groups separate traffic responsibilities.



No-IP • harshu.hopto.org CNAME → ELB hostname



AWS Security Groups

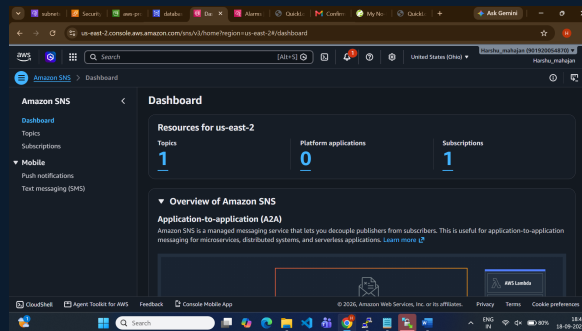
REQUEST PATH

Browser → harshu.hopto.org → ALB → tg1 → EC2

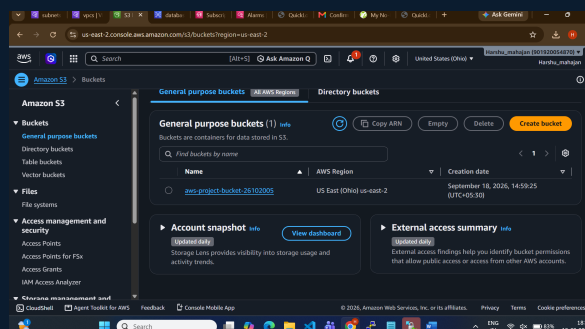
The supplied live evidence uses HTTP rather than HTTPS; HTTPS remains a production hardening item.

Notifications complete the monitoring loop.

Amazon SNS is configured as the notification component associated with the monitoring setup.



Amazon SNS dashboard / resources



SNS resource evidence

ALERTING WORKFLOW

CloudWatch alarm → SNS topic aws-project-1 → subscribed endpoint

The supplied console evidence shows the topic and a subscription.

The stack, at a glance.

Services documented through the project configuration and supplied AWS/application evidence.

EC2 Compute instances	ALB Traffic distribution	ASG Elastic capacity
VPC Network isolation	RDS MySQL database	S3 Object storage
CloudWatch Monitoring	SNS Notifications	AMI Reusable image
Launch Template Instance blueprint	NAT Gateway Outbound private access	No-IP Project hostname

PROJECT STACK

AWS • Linux • Web Application • MySQL • CloudWatch • SNS

Evidence-based project checks.

The supplied screenshots establish visible configuration and application reachability at capture time.

TC-01	Open public hostname	QuickLoan page loads	PASS
TC-02	Check ALB	lb1 active / Internet-facing	PASS
TC-03	Check listener	HTTP:80 → tg1	PASS
TC-04	Check Auto Scaling	asg-18sept • desired 1 • range 1–5	PASS
TC-05	Check launch template	lt1 version 1 available	PASS
TC-06	Check CloudWatch	CPU threshold alarms configured	PASS
TC-07	Check SNS	aws-project-1 topic / subscription	PASS
TC-08	Check DNS	CNAME points toward ALB hostname	PASS
TC-09	Open loan form	Form fields and controls render	PASS
TC-10	Check HTTPS	Secure connection	FOLLOW-UP

VALIDATION NOTE

Screenshots prove visible configuration; controlled load, database CRUD and notification delivery should be tested separately.

What this project demonstrates.

The evidence connects cloud infrastructure with a reachable application and operational controls.

01 Cloud deployment

Deployed a working loan application using multiple AWS infrastructure services.

02 Networking

Configured a custom VPC with public/private routing concepts, subnets and NAT.

03 Scalability

Implemented an Application Load Balancer with Auto Scaling and a launch template.

04 Managed services

Used RDS/MySQL for relational data and S3 for application assets.

05 Monitoring

Configured CloudWatch CPU alarms and an SNS notification component.

06 Hands-on operations

Worked directly with EC2, AMI, DNS, security groups and AWS console resources.

RESUME-READY SUMMARY

Deployed a scalable multi-tier QuickLoan application on AWS using VPC, EC2, Application Load Balancer, Auto Scaling, RDS/MySQL, S3, CloudWatch, SNS and DNS. Configured repeatable compute, load-balanced access and operational monitoring.

BUILD • DEPLOY • MONITOR • SCALE

Production hardening and next steps.

The current evidence is suitable for project demonstration; the following items define a practical improvement roadmap.

HTTPS

Enable an HTTPS listener with an SSL/TLS certificate on the Application Load Balancer.

Managed DNS

Move to Route 53 or another managed DNS solution if a production domain is required.

Security

Strengthen authentication, authorization, secrets handling and WAF protection.

Database

Add backup/restore validation and disaster-recovery procedures.

Observability

Add CloudWatch dashboards, application logs and centralized retention.

CI/CD

Connect GitHub to an automated build, test and deployment workflow.

Application quality

Correct static asset references and add automated application tests.

Scaling validation

Run controlled load tests to validate scale-out and scale-in behavior.

FINAL PROJECT MESSAGE

A structured AWS deployment can turn a simple web application into a demonstrable cloud project covering networking, compute, scaling, data, monitoring and